

Lecture 34: Natural Language Processing (NLP)

Sergei V. Kalinin

Text statistics: Bag of Words

- The Bag of Words model is a simple and commonly used method for representing text data in natural language processing (NLP).
- Treats each document as a "bag" of words, ignoring grammar, word order, and structure.
- Represents each document by counting the occurrences (frequency) of each word in the text.

Process:

1. **Tokenization:** Split each document into individual words (tokens).
2. **Vocabulary Creation:** Create a unique list (vocabulary) of all words across the dataset.
3. **Vectorization:** For each document, create a vector where each position corresponds to a word in the vocabulary, and the value at each position is the count (frequency) of that word in the document.

Text statistics: Bag of Words

1. Documents: ["The cat sat on the mat", "The dog sat on the log"]
2. Vocabulary: ["The", "cat", "sat", "on", "mat", "dog", "log"]
3. Vector Representation:
 - Document 1: [2, 1, 1, 1, 1, 0, 0]
 - Document 2: [2, 0, 1, 1, 0, 1, 1]

Pros:

- Simple and easy to implement.
- Effective for basic text representation in classification and clustering tasks.

Cons:

- Ignores word order, grammar, and semantics.
- Produces large, sparse vectors, especially for big vocabularies.
- Common words can overshadow the meaningful words

Term Frequency – Inverse Document Frequency

TF-IDF is a numerical statistic used to reflect the importance of a word in a document relative to a collection of documents (corpus).

Purpose: Enhances the Bag of Words model by giving less weight to common words and more weight to unique or informative words, improving relevance in text analysis.

Components:

- Term Frequency (TF): Measures how often a word appears in a document.
Document Frequency (IDF): Measures how unique a word is across all documents (Total number of documents/Number of documents containing the word)
- TF-IDF Score: Calculated by multiplying TF and IDF, giving a higher score to unique, informative words.

Term Frequency – Inverse Document Frequency

Process:

1. Tokenization: Split each document into words.
2. Calculate TF for each word in each document.
3. Calculate IDF for each word across the entire corpus.
4. Compute TF-IDF score for each word in each document.

Example:

- Documents: ["The cat sat on the mat", "The dog sat on the log"]
- Word "sat" appears once in each document, so it has a lower IDF (more common across documents), reducing its TF-IDF score.
- Rare words like "cat" or "dog" have a higher IDF, boosting their TF-IDF scores.

Pros:

- Highlights informative words by down-weighting common words.
- Improves relevance in tasks like document classification and information retrieval.

Cons:

- Does not capture word order or semantics.
- Creates large, sparse vectors with big vocabularies.
- Provides a more nuanced representation than Bag of Words by emphasizing distinctive terms across documents, which is useful for tasks requiring word importance differentiation.

Term Frequency – Inverse Document Frequency

Creation:

- Transform each document into a vector of TF-IDF scores, where each element represents a word in the vocabulary.
- Compile these vectors into a TF-IDF matrix, with documents as rows and words as columns, making the text data ready for machine learning.

Input for Machine Learning Models:

- Use TF-IDF features as inputs for algorithms such as Naive Bayes, Support Vector Machines, and Logistic Regression in text classification tasks.
- Train models to classify documents, detect sentiment, or categorize topics based on the TF-IDF vector representation.
- Similarity Measurement: Use TF-IDF vectors to calculate cosine similarity between documents to find related content or cluster similar documents.
- Dimensionality Reduction: Reduce the TF-IDF matrix's dimensionality with techniques like PCA or LSA for efficient processing and better model performance.
- Information Retrieval: Rank documents based on their TF-IDF scores, with higher scores indicating more relevant matches to search queries.

There must be order: N-gram models

N-gram models *assume* each word (event) depends only on the previous $n-1$ words (events):

Unigram model: $P(w^{(1)} \dots w^{(N)}) = \prod_{i=1}^N P(w^{(i)})$

Bigram model: $P(w^{(1)} \dots w^{(N)}) = \prod_{i=1}^N P(w^{(i)} | w^{(i-1)})$

Trigram model: $P(w^{(1)} \dots w^{(N)}) = \prod_{i=1}^N P(w^{(i)} | w^{(i-1)}, w^{(i-2)})$

NB: Independence assumptions where the n -th event in a sequence depends only on the last $n-1$ events are called **Markov assumptions (of order $n-1$)**.

Where do N-grams come from?

Where do we get the parameters of our model (actual probabilities) from?

$$P(w(i) = \text{'the'} \mid w(i-1) = \text{'on'}) = ???$$

We need (a large amount of) text as training data to estimate the parameters of a language model.

The most basic parameter estimation technique: relative frequency estimation (frequency = counts)

$$P(w(i) = \text{'the'} \mid w(i-1) = \text{'on'}) = C(\text{'on the'}) / C(\text{'on'})$$

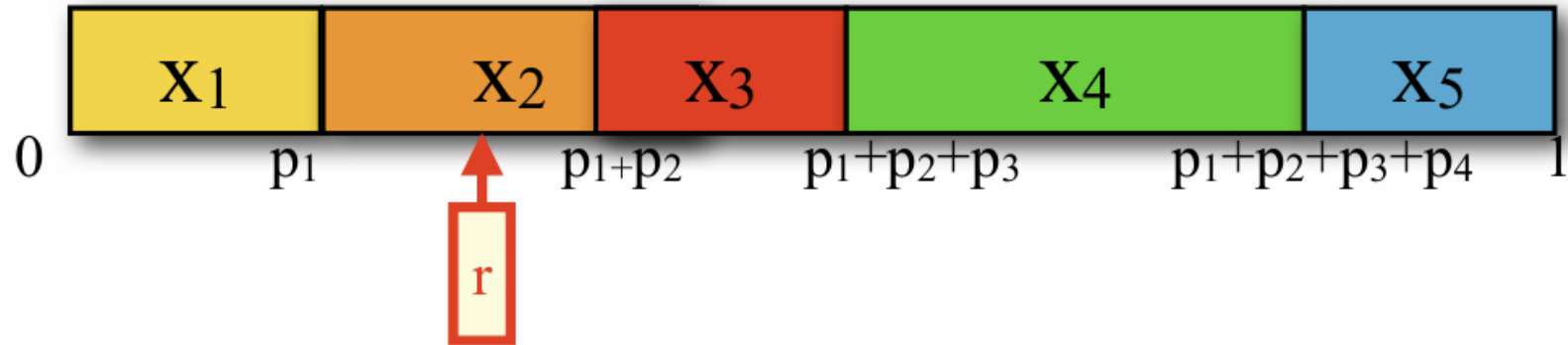
Also called Maximum Likelihood Estimation (MLE)

$C(\text{'on the'})$ [or $f(\text{'on the'})$ for frequency]: How often does 'on the' appear in the training data?

Generating text from N-gram model

That is, how do you sample from a distribution $P(X | Y=y)$?

- Assume X has N possible outcomes (values): $\{x_1, \dots, x_N\}$ and $P(X=x_i | Y=y) = p_i$
- Divide the interval $[0,1]$ into N smaller intervals according to the probabilities of the outcomes
- Generate a random number r between 0 and 1.
- Return the x_i whose interval the number is in.



Generating the Wall Street Journal

unigram: Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

bigram: Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

trigram: They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Generating Shakespeare

Unigram	• To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have • Every enter now severally so, let • Hill he late speaks; or! a more to leg less first you enter • Are where exeunt and sighs have rise excellency took of.. Sleep knave we. near; vile like
Bigram	• What means, sir. I confess she? then all sorts, he is trim, captain. • Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. • What we, hath got so she that I rest and sent to scold and nature bankrupt, nor the first gentleman? • Enter Menenius, if it so many good direction found'st thou art a strong upon command of fear not a liberal largess given away, Falstaff! Exeunt
Trigram	• Sweet prince, Falstaff shall die. Harry of Monmouth's grave. • This shall forbid it should be branded, if renown made it empty. • Indeed the duke; and had a very good friend. • Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.
Quadrigram	• King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; • Will you not tell me who I am? • It cannot be but so. • Indeed the short and the long. Marry, 'tis a noble Lepidus.

The Shakespeare corpus has $N=884,647$ word tokens for a vocabulary of $V=29,066$ word types.

Shakespeare used 300,000 bigram types out of $V^2=844$ million possible bigram types. 99.96% of possible bigrams don't occur in this corpus.

Corollary: A relative frequency estimate based on this corpus assigns non-zero probability to only 0.04% of possible bigrams

That percentage is even lower for trigrams, 4-grams, etc. 4-grams look like Shakespeare because they are Shakespeare!

Text Classification

- Assigning subject categories, topics, or genres
 - Spam detection
 - Authorship identification (who wrote this?)
 - Language Identification (is this Portuguese?)
 - Sentiment analysis
 - ...
-
- *Input:*
 - a document d
 - a fixed set of classes $C = \{c_1, c_2, \dots, c_J\}$
 - *Output:* a predicted class $c \in C$

Text Classification

- Rules based on combinations of words or other features
 - spam: black-list-address OR (“dollars” AND “have been selected”)
- Accuracy can be high
 - In very specific domains
 - If rules are carefully refined by experts

But:

- building and maintaining rules is expensive
- they are too literal and specific: "high-precision, low-recall"

Text Classification

- *Input:*
 - a document d
 - a fixed set of classes $C = \{c_1, c_2, \dots, c_J\}$
 - A training set of m hand-labeled documents $(d_1, c_1), \dots, (d_m, c_m)$
 - *Output:*
 - a learned classifier $\gamma: d \rightarrow c$
 - Many kinds of classifiers!
 - Naïve Bayes (this lecture)
 - Logistic regression
 - Neural networks
 - k -nearest neighbors
 - ...
- We can also use pretrained LLMs!
- Fine-tuned as classifiers
 - Prompted to give a classification

Naïve Bayes

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' Theorem. It assumes independence between features, simplifying computation (hence "naive").

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

- $P(C|X)$: Posterior probability of class C given data X .
- $P(X|C)$: Likelihood of data X for class C .
- $P(C)$: Prior probability of class C .
- $P(X)$: Evidence, a normalizing constant.

2. Independence Assumption:

- Assumes all features $X_1, X_2, \dots, X_n$ are conditionally independent given class C :

$$P(X|C) = P(X_1|C) \cdot P(X_2|C) \cdot \dots \cdot P(X_n|C)$$

Naïve Bayes

- From training corpus, extract *Vocabulary*
- Calculate $P(c_j)$ terms
 - For each c_j in C do
 - $docs_j \leftarrow$ all docs with class = c_j
- Calculate $P(w_k | c_j)$ terms
 - $Text_j \leftarrow$ single doc containing all $docs_j$
 - For each word w_k in *Vocabulary*
 - $n_k \leftarrow$ # of occurrences of w_k in $Text_j$

$$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$

$$P(w_k | c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

Naïve Bayes: Advantages

- Very Fast, low storage requirements
- Work well with very small amounts of training data
- Robust to Irrelevant Features
 - Irrelevant Features cancel each other without affecting results
- Very good in domains with many equally important features
 - Decision Trees suffer from *fragmentation* in such cases – especially if little data
- Optimal if the independence assumptions hold: If assumed independence is correct, then it is the Bayes Optimal Classifier for problem
- A good dependable baseline for text classification

Naïve Bayes: Advantages

- Naïve bayes classifiers can use any sort of feature
 - URL, email address, dictionaries, network features
- But if, as in the previous slides
 - We use **only** word features
 - we use **all** of the words in the text (not a subset)
- Then
 - Naïve bayes has an important similarity to language modeling.

But what about meaning?

N-gram or text classification methods we've seen so far

- Words are just strings (or indices w_i in a vocabulary list)
- That's not very satisfactory!

Relations between words

- Synonymy
- Antonymy
- Similarity
- Relatedness
- Connotation

word1	word2	similarity
vanish	disappear	9.8
behave	obey	7.3
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

SimLex-999 dataset (Hill et al., 2015)

But what about meaning?

"The meaning of a word is its use in the language"

Ludwig Wittgenstein

- One way to define "usage": words are defined by their environments (the words around them)
- **If A and B have almost identical environments we say that they are synonyms.**

Zellig Harris (1954):

Word embeddings

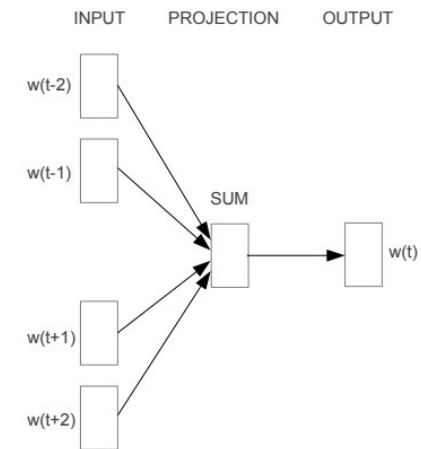
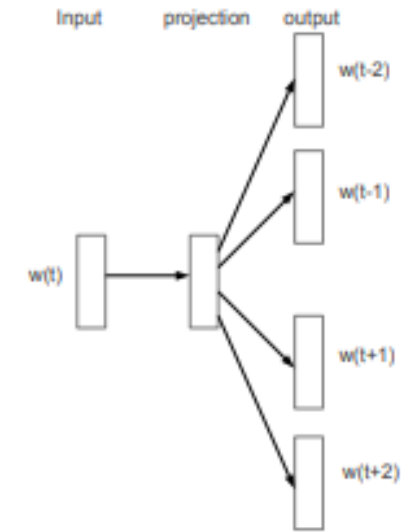
- Called an "embedding" because it's embedded into a space (see textbook)
- The standard way to represent meaning in NLP
- Fine-grained model of meaning for similarity
- **tf-idf**
 - Information Retrieval workhorse!
 - A common baseline model
 - **Sparse** vectors
 - Words are represented by (a simple function of) the **counts** of nearby words
- **Word2vec**
 - **Dense** vectors
 - Representation is created by training a classifier to **predict** whether a word is likely to appear nearby

Word embeddings

- “Neural Language Model”-inspired models
 - Word2vec (skipgram, CBOW), GloVe
- Singular Value Decomposition (SVD)
 - A special case of this is called LSA – Latent Semantic Analysis
- Alternative to these "static embeddings":
 - Contextual Embeddings (ELMo, BERT)
 - Compute distinct embeddings for a word in its context
 - Separate embeddings for each token of a word
- Word2vec (Mikolov et al): <https://code.google.com/archive/p/word2vec/>
- GloVe (Pennington, Socher, Manning): <http://nlp.stanford.edu/projects/glove/>

Word embeddings: Word2Vec

- Popular embedding method
- Very fast to train
- Code available on the web
- Idea: **predict** rather than **count**
- Word2vec provides various options. We'll do:
 - **skip-gram with negative sampling (SGNS)**



CBOW

<https://medium.com/geekculture/word-embeddings-in-ai-10a9e430cb59>

https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf

Word embeddings: Word2Vec

1. Treat the target word t and a neighboring context word c as **positive examples**.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as the embeddings

Start with V random d -dimensional vectors as initial embeddings

Train a classifier based on embedding similarity

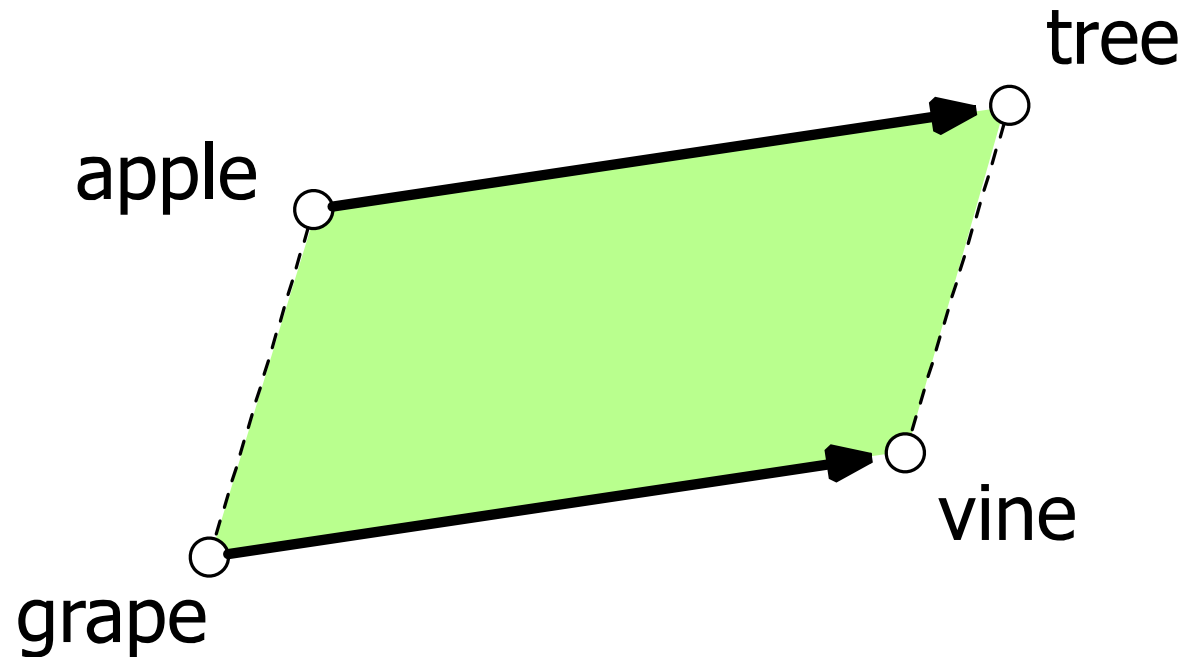
- Take a corpus and take pairs of words that co-occur as positive examples
- Take pairs of words that don't co-occur as negative examples
- Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
- Throw away the classifier code and keep the embeddings.

Word embeddings: Word2Vec

- **Small windows** ($C = \pm 2$) : nearest words are syntactically similar words in same taxonomy
 - *Hogwarts* nearest neighbors are other fictional schools
 - *Sunnydale, Evernight, Blandings*
- **Large windows** ($C = \pm 5$) : nearest words are related words in same semantic field
 - *Hogwarts* nearest neighbors are Harry Potter world:
 - *Dumbledore, half-blood, Malfoy*

Analogical relations

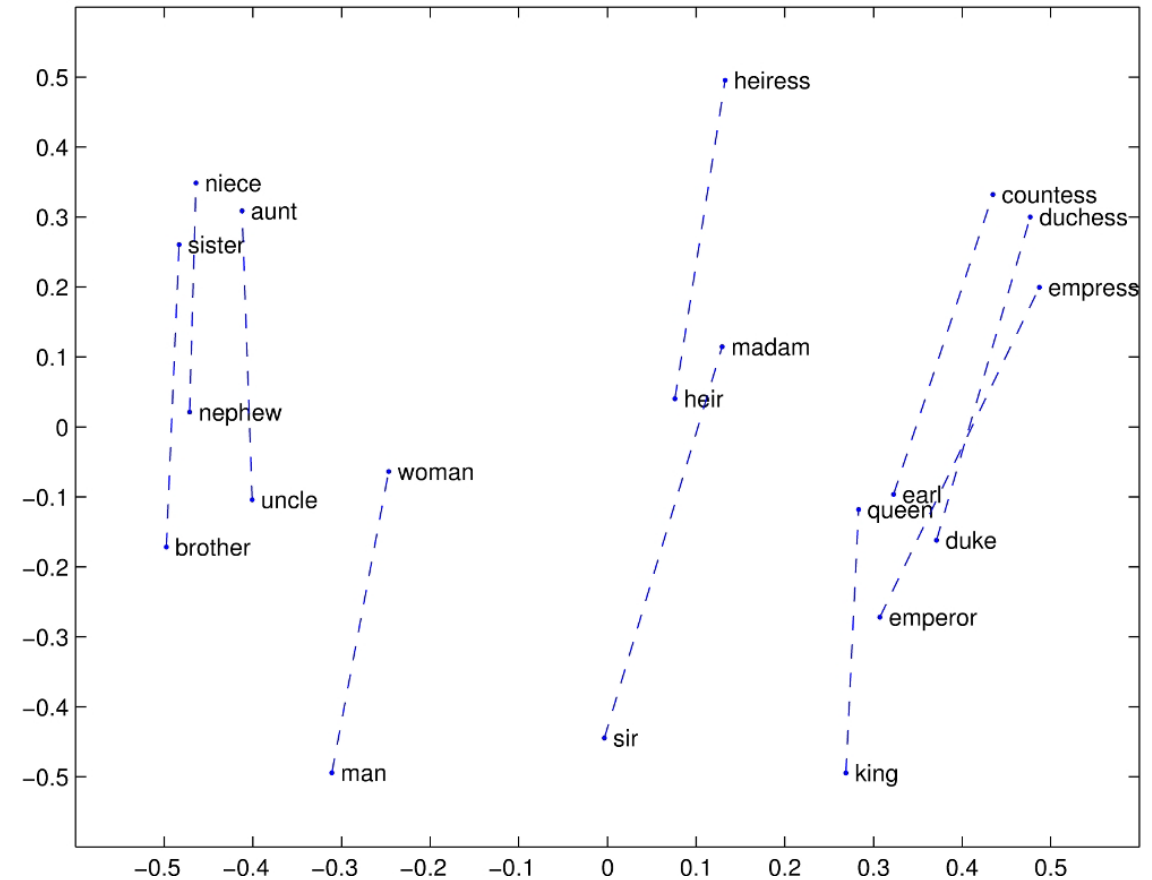
- The classic parallelogram model of analogical reasoning (Rumelhart and Abrahamson 1973)
- To solve: *"apple is to tree as grape is to _____"*
- Add tree – apple to grape to get *vine*



Analogical relations

- The parallelogram method can solve analogies with both sparse and dense embeddings (Turney and Littman 2005, Mikolov et al. 2013b)
- *king* – *man* + *woman* is close to *queen*
- *Paris* – *France* + *Italy* is close to *Rome*
- For a problem $a:a^*:b:b^*$, the parallelogram method is:

$$\hat{b}^* = \operatorname{argmax}_x \operatorname{distance}(x, a^* - a + b)$$



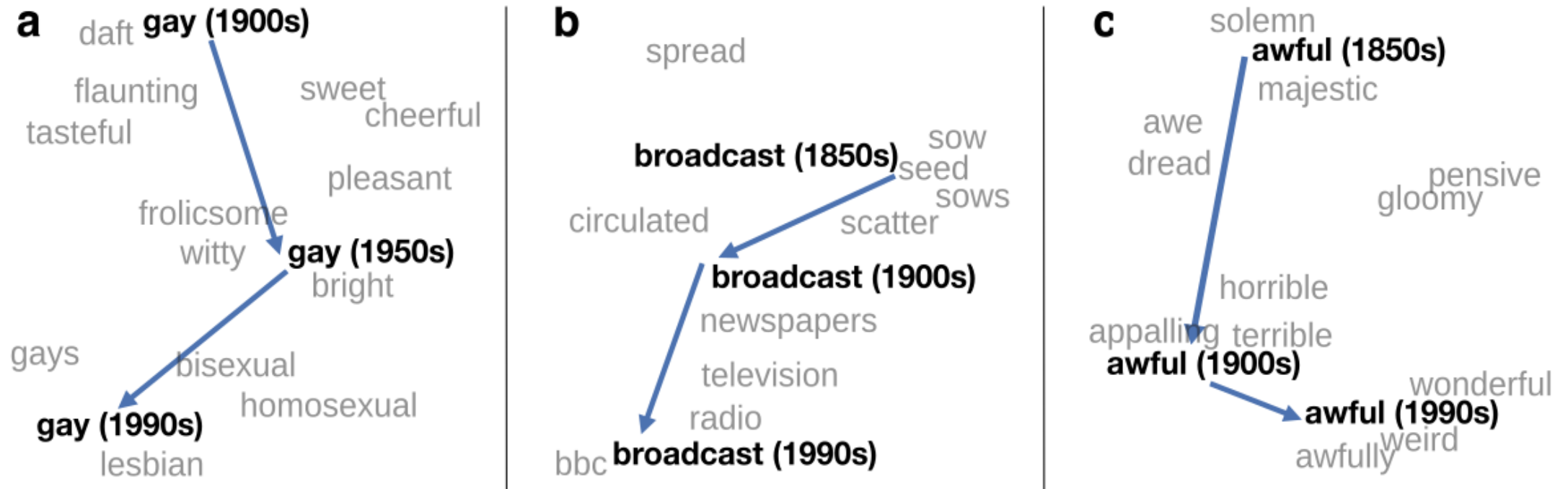
Caveats

- It only seems to work for frequent words, small distances and certain relations (relating countries to capitals, or parts of speech), but not others. (Linzen 2016, Gladkova et al. 2016, Ethayarajh et al. 2019a)
- Understanding analogy is an open area of research (Peterson et al. 2020)

Historical semantics

Train embeddings on different decades of historical text to see meanings shift

~30 million books, 1850-1990, Google Books data



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

Cultural Biases

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *NeurIPS*, pp. 4349-4357. 2016.

- Ask “Paris : France :: Tokyo : x”
 - x = Japan
- Ask “father : doctor :: mother : x”
 - x = nurse
- Ask “man : computer programmer :: woman : x”
 - x = homemaker

Algorithms that use embeddings as part of e.g., hiring searches for programmers, might lead to bias in hiring

Cultural Biases

Garg, N., Schiebinger, L., Jurafsky, D., and Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. *Proceedings of the National Academy of Sciences* 115(16), E3635–E3644.

- Compute a **gender or ethnic bias** for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities
 - Embeddings for **competence** adjective (*smart, wise, brilliant, resourceful, thoughtful, logical*) are biased toward men, a bias slowly decreasing 1960-1990
 - Embeddings for **dehumanizing** adjectives (barbaric, monstrous, bizarre) were biased toward Asians in the 1930s, bias decreasing over the 20th century.
- These match the results of old surveys done in the 1930s